NCL Fall 2026:
Gymnasium Log Analysis, Custom File Format

CTF Walk-through & Technical Analysis Guide

Glenn R. Harshman

- ***Category:*** *Forensics, Miscellaneous*
- ***Difficulty:*** *Hard*
- ***Points:*** *100*
- ***Core Concepts:*** *Digital Forensics, Blue Team*
  Attack Pattern Recognition
  Log Parsing and Data Manipulation
  Timeline Analysis & Incident Reconstruction
  Obfuscation and Decoding

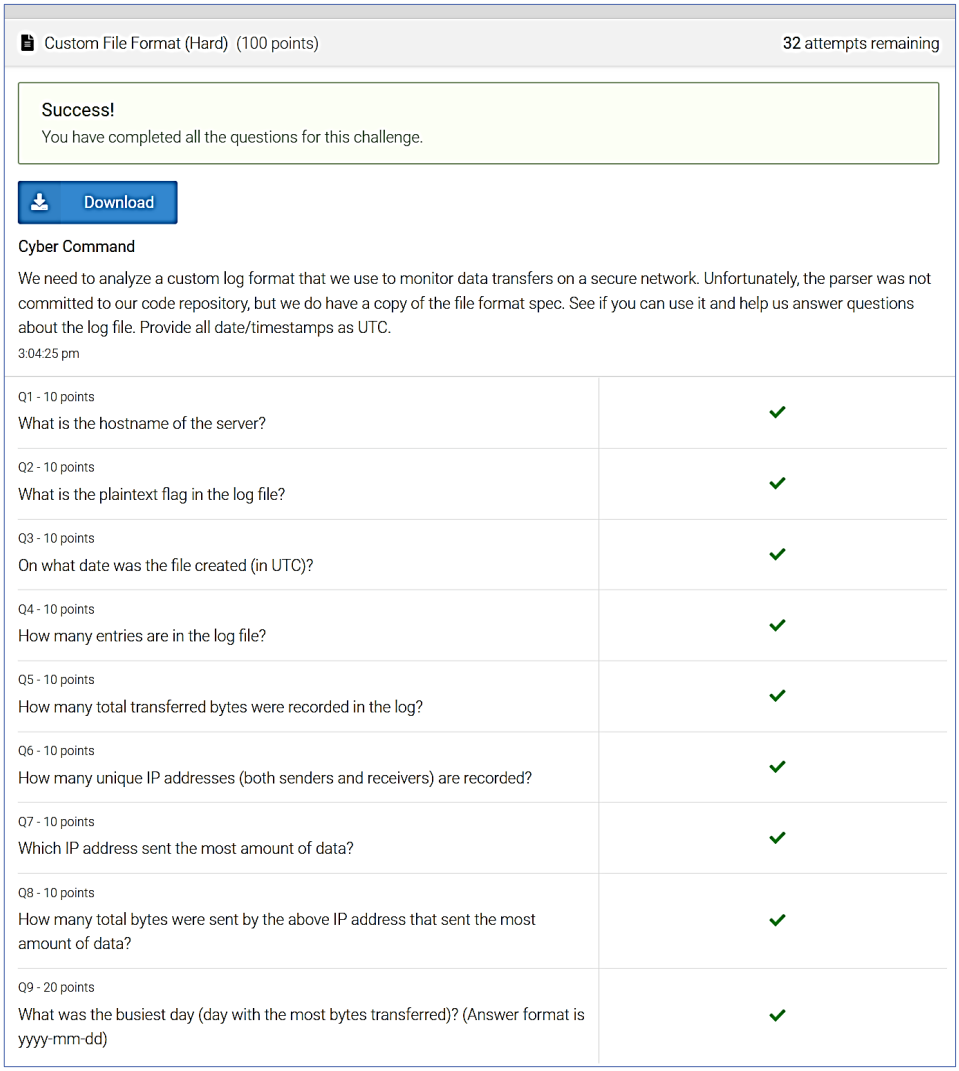
1. Challenge Context & Initial Triage

Log analysis refers to the process of reviewing and interpreting system, network, or application logs to identify patterns, detect threats, and gain actionable insights. It plays a key role in security incident detection, troubleshooting, and compliance monitoring. CTF log files are often quite large, requiring players to use command-line tools or scripting to isolate the flag or malicious activity.

In this exercise, the log file was designed to record network traffic data in an efficient manner, utilizing raw binary data to save space compared to traditional text files. This was not done to intentionally obfuscate the data, but the effect on log analysis is the same, if you do not have a custom log reader for the file.

The exercise provides an example log file ('**`Custom File Format.sky`**'), and the specifications for the file's structure. We need to analyze this custom log file that was used to monitor data transfers on a secure network. Unfortunately, the log reader was not given to us; only the file specification was given. All date/time stamps provided as UTC. Answers should also be in UTC format. The final complication is that the file data is stored in "Big Endian" format, while most computer systems default to "Little Endian" format. What that means is that every two-byte group is reversed, but it is a simple matter to manage the byte order.

If we were going to analyze the log file format regularly and repeatedly, we would program a script or automated solution. Since this is a one-time analysis, we will use regular Linux command line tools.



Specification For The File Header:

The SKY header begins at byte offset 0 and is comprised of the following fields in order with no padding. Because the file is saved in binary format, each field must be converted to its appropriate type, e.g., hex, decimal, or ASCII, depending on the individual fields' purposes.

- **Magic Bytes = 8 bytes or 64 bits Hex**

The magic bytes field is an 8-byte unique sequence used to identify the file as using the SKY format. All SKY files begin with the same hex ("0x") sequence: "91534B590D0A1A0A". We can see this signature in our example file using the Linux **xxd** command.

The Linux **xxd** command is a utility used to inspect files or standard input as binary bits, hex representations, and ASCII characters. It is widely used by system administrators, developers, and security analysts for debugging binaries, data analysis, and file patching. It has many flags to control the output, but in this example, we only use four:

- **-s** Start byte (or "skip" bytes)
- **-l** Length in bytes to output
- **-b** Binary output
- **-c** Columns to display per line

```
nicegoat@Goatyard:~$ xxd -s 0 -l 8 custom.sky
00000000: 9153 4b59 0d0a 1a0a                      .SKY....

nicegoat@Goatyard:~$ xxd -s 0 -l 8 -b -c 8 custom.sky
00000000: 10010001 01010011 01001011 01011001 00001101 00001010 00011010 00001010  .SKY....
```

There is another Linux command, **hexdump**, which is also useful for this same type of analysis. However, **hexdump** defaults to reading bytes per the system default on which the command is run, which for most systems is "Little Endian" format. You can use it, but you have to manage the byte order by only reading one (1) byte at a time using the **-e** flag and a format string. The **-v** flag forces verbose mode, bypassing **hexdump**'s desire to compress the output. The **-s** flag is the same as with **xxd**, but the length flag changes to **-n**.

The disadvantage of **hexdump** is that it does not natively display the ASCII conversion along with the byte readout. However, **hexdump**'s output can be piped to **xxd** for conversion (use **-b** for binary and **-r** for ASCII). Notice the "unprintable" characters in the ASCII output.

```
nicegoat@Goatyard:~$ hexdump -s 0 -n 8 -v -e '8/1 "%02x"' custom.sky
91534b590d0a1a0a
nicegoat@Goatyard:~$ hexdump -s 0 -n 8 -v -e '8/1 "%02x"' custom.sky | xxd -b -c 8
00000000: 00111001 00110001 00110101 00110011 00110100 01100010 00110101 00111001  91534b59
00000008: 00110000 01100100 00110000 01100001 00110001 01100001 00110000 01100001  0d0a1a0a
nicegoat@Goatyard:~$ hexdump -s 0 -n 8 -v -e '8/1 "%02x"' custom.sky | xxd -r -p
SKY
```

- **Version Number = 1 byte or 8 bits Integer/Decimal**

The version field is a single byte. The specification document says that all SKY files use version 1 at this time, so this field should be "0x00000001" in hex or "1" decimal. The field starts at byte 8 (i.e., skipping bytes 0 through 7), and the length of the field is one (1) byte.

```
nicegoat@Goatyard:~$ xxd -s 8 -l 1 -b -c 8 custom.sky
00000008: 00000001
nicegoat@Goatyard:~$ hexdump -s 8 -n 1 -v -e '8/1 "%02x"' custom.sky
01
```

- **Creation Time: = 4 bytes or 32 bits Timestamp**

The creation timestamp is a single timestamp used to denote the start of data collection within the log file. It starts at byte 9, and the length is four (4) bytes. It is in standard Linux long integer format, which is the number of seconds since January 1st, 1970 at 12:00:00AM. The integer must be converted to UTC time to be understandable to humans. To do that, we put the hex value through an evaluation function **`$((x))`** and **`echo`** it to the screen (there are other ways to convert hex to decimal, but this is the easiest). Then we put the decimal into the **`date`** command to convert it to a human-readable date & time. The **`-d`** flag tells the **`date`** command that the input is a string, but the '**@**' tells the command that the input is a decimal number. The **`-u`** flag tells the **`date`** command to convert to UTC time, not local time.

```
nicegoat@Goatyard:~$ hexdump -s 9 -n 4 -v -e '1/1 "%02x"' custom.sky
5a9a2bd0
nicegoat@Goatyard:~$ echo $(("0x5a9a2bd0"))
1520053200
nicegoat@Goatyard:~$ date -u -d @1520053200
Sat Mar  3 05:00:00 UTC 2018
nicegoat@Goatyard:~$ date -u -d @1520053200 "+%Y-%m-%d %H:%M:%S"
2018-03-03 05:00:00
```

- **Hostname Length = 4 bytes or 32 bits Integer/Decimal**

The hostname length is a single decimal integer used to denote the number of bytes needed for the hostname. It starts at byte 13, and the length is four (4) bytes. If no host is specified, this value should be 0x00000000. In this case, the value is 0x0000000e, which converts to 14 bytes in decimal (base-10).

```
nicegoat@Goatyard:~$ hexdump -s 13 -n 4 -v -e '1/1 "%02x"' custom.sky
0000000e
nicegoat@Goatyard:~$ echo $(("0x0000000e"))
14
```

- **Hostname = # Bytes in preceding value ASCII**

The hostname is a dynamic length string used to identify the name of the host on which the log file was created. The byte-length of the hostname is the value from the previous field, in this case, 14 bytes. Therefore, the field starts at byte 17, and the length is 14 bytes. The server name needs to be converted to ASCII characters to be human-readable. In this case, **xxd** is the easier and shorter command to run.

```
nicegoat@Goatyard:~$ xxd -s 17 -l 14 -c 14 custom.sky
00000011: 736b 792d 7365 7276 6572 2d37 3131  sky-server-711
nicegoat@Goatyard:~$ hexdump -s 17 -n 14 -v -e '1/1 "%02x"' custom.sky
736b792d7365727665722d373131
nicegoat@Goatyard:~$ hexdump -s 17 -n 14 -v -e '1/1 "%02x"' custom.sky | xxd -r -p
sky-server-711
```

- **Flag Length = 4 bytes or 32 bits Integer/Decimal**

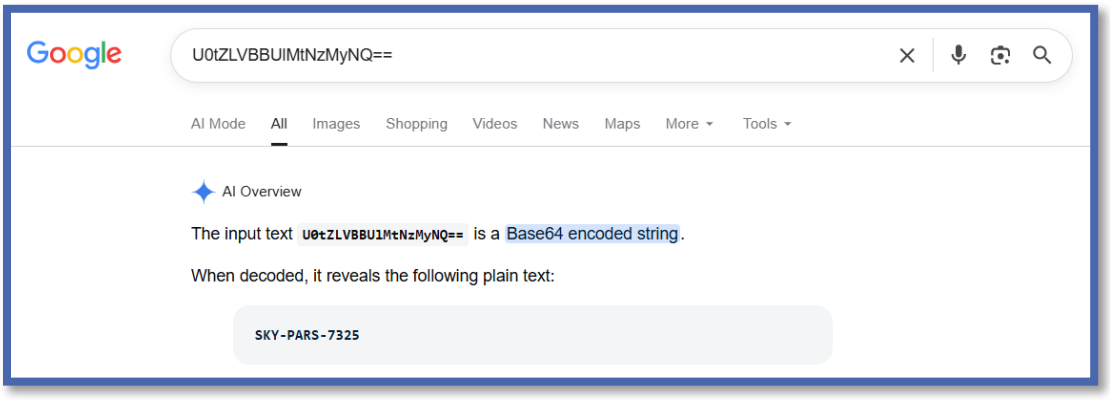
The flag length is a decimal integer used to denote how many bytes long is the flag value. This value is used to determine how many bytes to read for the following field. It starts at byte 31, and the length is four (4) bytes. If no flag is specified, this value should be 0x00000000. In this case, the value is 0x00000014, or 20 bytes.

```
nicegoat@Goatyard:~$ xxd -s 31 -l 4 -c 14 custom.sky
0000001f: 0000 0014
nicegoat@Goatyard:~$ echo $(("0x00000014"))
20
```

- **Flag Value = # Bytes in preceding field ASCII Base64**

The flag is a dynamic length string used to specify a flag value for the log file. The byte-length of the flag is the value from the previous field. It starts at byte 35, and for this file, the length is 20 bytes. This field can be used to store both encoded/encrypted flags as well as plaintext flags. If the flag appears to be encoded, you have to figure out how to decode it. In this case, I put the ASCII conversion in Google AI, and it suggested that it was encoded with **base64**.

```
nicegoat@Goatyard:~$ xxd -s 35 -l 20 -p custom.sky
5530745a4c564242556c4d744e7a4d794e513d3d
nicegoat@Goatyard:~$ xxd -s 35 -l 20 -p custom.sky | xxd -r -p
U0tZLVBBUlMtNzMyNQ==
nicegoat@Goatyard:~$ xxd -s 35 -l 20 -p custom.sky | xxd -r -p | base64 --decode
SKY-PARS-7325
```



- **Number of records = 4 bytes or 32 bits Integer/Decimal**

The last header field is the number of log entries in the body. This field is a single decimal integer used to denote the number of records to follow. For this file, the field starts at byte 55, and the length is four (4) bytes. In this case, the value is 0x000000a2, or 162 log record entries.

```
nicegoat@Goatyard:~$ xxd -s 55 -l 4 -p custom.sky
000000a2
nicegoat@Goatyard:~$ echo $((0xa2))
162
```

Specification For The File Body:

The body is a sequence of items, each with 4 required fields, and each field is four bytes long. All items are written in chronological order without any padding between items. Each item contains the following fields:

- **Source IP Address = 4 bytes or 32 bits Four integers 0-255**

The IPv4 address of the sender of the data stream. Represented as four short integers between 0 and 255, which means 8 bits each ($2^8 = 256$). The decimal "." is not represented in the data. It is a convention to delimit the four integers, thus making the IP address more readable. For this file, the first body data record starts at byte 59, and the length is

four (4) bytes. In this case, the value is 0x32c7ccb7, which must be broken into two-byte chunks as 0x(32 c7 cc b7). These values convert to decimal as 50, 199, 204, and 183.

```
nicegoat@Goatyard:~$ xxd -s 59 -l 4 -p custom.sky
32c7ccb7
nicegoat@Goatyard:~$ echo "$((0x32)).$((0xc7)).$((0xcc)).$((0xb7))"
50.199.204.183
```

- **Destination IP Address = 4 bytes or 32 bits Four integers 0-255**

The IPv4 address of the destination of the data stream. Represented as four short integers between 0 and 255, just like the source IP address above. For this file, the second body data field starts at byte 63, and the length is four (4) bytes. In this case, the value is 0xe5d415d4, which must be broken into two-byte chunks as 0x(e5 d4 15 d4). These values convert to decimal as 229, 212, 21, and 212.

```
nicegoat@Goatyard:~$ xxd -s 63 -l 4 -p custom.sky
e5d415d4
nicegoat@Goatyard:~$ echo "$((0xe5)).$((0xd4)).$((0x15)).$((0xd4))"
229.212.21.212
```

- **Date/Time Stamp: = 4 bytes or 32 bits Timestamp**

The timestamp when the data stream was initiated. Represented as one long integer. For this file, the third body data field starts at byte 67, and the length is four (4) bytes. In this case, the value is 0x5a9a44f5, which converts to decimal 1520059637. Again, this is the number of seconds since Jan. 1st, 1970 at 12:00:00AM.

```
nicegoat@Goatyard:~$ xxd -s 67 -l 4 -p custom.sky
5a9a44f5
nicegoat@Goatyard:~$ echo "$((0x5a9a44f5))"
1520059637
```

- **Transferred Bytes: = 4 bytes or 32 bits Integer/Decimal**

The number of bytes transferred in the data stream. Represented as one long integer. For this file, the fourth body data field starts at byte 71, and the length is four (4) bytes. In this case, the value is 0x000006c3, which converts to decimal 1731.

```
nicegoat@Goatyard:~$ xxd -s 71 -l 4 -p custom.sky
000006c3
nicegoat@Goatyard:~$ echo "$((0x000006c3))"
1731
```

CONVERTING THE FILE BODY TO FACILITATE ANALYSIS:

Because the body is composed of records of fixed and predictable length, this makes it very easy to convert the entire body into human-readable format using simple Linux commands. We use the **`tail`** command because we want everything from after the header to the end of the file. The **`-c +60`** flags mean to read every byte one at a time, starting with the 60th byte from the beginning (notice that this is different than saying start at byte 59). From there, we pipe the bytes to the **`hexdump`** command to convert the binary bytes to hex. Next, we pipe to the **`awk`** command to take advantage of its ability to create and use functions on the fly. We create a function to convert hex inputs to formatted IP addresses, and then we call that function as we convert columns 1 and 2. Columns 3 and 4 are simply converted to decimal numbers using the **`strtonum()`** function. The **`print`** command automatically puts a space delimiter between the fields. Finally, we redirect (**`>`**) the standard output to a new file called **`entries.txt`**.

```
nicegoat@Goatyard:~$ tail -c +60 custom.sky | hexdump -v -e '4/1 "%02x" " " 4/1 "%02x" " " 4/1 "%02x" " " 4/1 "%02x" "\n"' | awk '
function hex2ip(hex) {
    return strtonum("0x" substr(hex,1,2)) "." \
           strtonum("0x" substr(hex,3,2)) "." \
           strtonum("0x" substr(hex,5,2)) "." \
           strtonum("0x" substr(hex,7,2))
}
NF==4 {
    print hex2ip($1), hex2ip($2), strtonum("0x"$3), strtonum("0x"$4)
}' > entries.txt


nicegoat@Goatyard:~$ cat entries.txt | head -20
50.199.204.183 229.212.21.212 1520059637 1731
224.58.7.136 229.212.21.212 1520073403 1927
247.205.80.23 184.238.0.32 1520079371 7557
143.110.35.240 175.134.52.113 1520088683 502
247.205.80.23 77.255.37.60 1520094216 208
175.134.52.113 77.255.37.60 1520096636 5103
88.171.101.239 177.132.224.221 1520120038 5393
137.216.107.80 229.212.21.212 1520129839 2790
77.255.37.60 88.171.101.239 1520140792 5862
247.205.80.23 224.58.7.136 1520154901 3685
23.101.47.99 247.205.80.23 1520165235 2337
177.132.224.221 143.110.35.240 1520172881 4275
229.212.21.212 247.205.80.23 1520186070 5567
175.134.52.113 137.216.107.80 1520196890 1542
137.216.107.80 88.171.101.239 1520217741 1083
247.205.80.23 77.255.37.60 1520218700 6547
184.238.0.32 77.255.37.60 1520240749 9674
175.134.52.113 88.171.101.239 1520268895 749
88.171.101.239 50.199.204.183 1520273772 1962
229.212.21.212 171.233.106.117 1520282715 509
```

Now that we completely understand the structure of the file, have dissected the header and discovered the two missing field lengths, and we have created a new file which holds the individual log record entries --- we are ready to proceed with the CTF questions.

2. Challenge Question 1: What is the hostname of the server? (10 Points)

This question was answered above in Section 1, at the top of page 4.

Verified Answer: "`sky-server-711`"

3. Challenge Question 2: What is the plaintext flag in the log file? (10 Points)

This question was answered above in Section 1, at the bottom of page 4 and the top of page 5.

Verified Answer: "`SKY-PARS-7325`"

4. Challenge Question 3: On what date was the file created (in UTC)? (10 Points)

This question was answered above in Section 1, in the middle of page 3.

Verified Answer: "`2018-03-03`"

5. Challenge Question 4: How many entries are in the log file? (10 Points)

This question was answered above in Section 1, in the middle of page 5.

Verified Answer: "`162`"

6. Challenge Question 5: How many total transferred bytes were recorded in the log? (10 Points)

Finally, a question that was not answered while we were conducting initial triage!

In section 1, we recognized that the file body is very regular and repeatable. Each record is four (4) fields of four (4) bytes each, so we went ahead and extracted the records and saved them in text format for easier analysis.

Column 4 is the number of bytes transferred for each record. We simply need to pass our new text file through **awk** and sum the decimals in column 4.

```
nicegoat@Goatyard:~$ cat entries.txt | awk '{sum += $4} END {print sum}'
811167
```

Verified Answer: "811167"

7. Challenge Question 6: How many unique IP addresses (both senders and receivers) are recorded? (10 Points)

We need the IP addresses from column 1 and from column 2 to be combined into a single column so that we can manipulate them as one group. The easiest way to do this is to write column 1 out to a new file, and then write column 2 out to the same file (i.e., an append operation). We do that using another redirection operator (**>>**), which appends the data to an existing file (or creates the file, if it does not yet exist). Then, we can run our new file through **sort**, **uniq**, and **wc -l**. This sorts the IP addresses, which is required by **uniq** to remove all duplicates, and then **wc -l** counts the number of lines remaining.

```
nicegoat@Goatyard:~$ cat entries.txt | awk '{print $1}' >> ipaddrs.txt
nicegoat@Goatyard:~$ cat entries.txt | awk '{print $2}' >> ipaddrs.txt
nicegoat@Goatyard:~$ cat ipaddrs.txt | sort | uniq | wc -l
13
```

Verified Answer: "13"

8. Challenge Question 7: Which IP address sent the most amount of data? (10 Points)

In this question, we need to create an *associative array* (aka "dictionary" in Python or "map" in C++) in order to keep track of bytes per IP address. Associative arrays are known as "*key-value pairs*". In Excel, this would be like creating a Pivot Table to sum transferred bytes by IP address.

In order to do this, we need to isolate columns 1 (Source IP Address) and 4 (transferred bytes). Then we can have **`awk`** create our associative array. Each time that **`awk`** encounters a new IP address, it adds that IP address as a new key in the array. Then it adds the bytes to that key. If the key already exists, it adds the bytes to the value that was already present in the array. Then we have **`awk`** run a **`for`** loop to cycle through the array and send the key-value pairs to standard output. However, because we are interested in the largest number of bytes transferred, we need the values on the left and the keys on the right, so that our **`sort`** command will work properly. We include the **`-n`** and **`-r`** flags with **`sort`** so that the transferred bytes will be sorted numerically (not *alpha*-numerically) and arranged in descending order, placing our answer at the top of the list (answer = the ***key*** on the right side of the first line).

```
nicegoat@Goatyard:~$ cat entries.txt | awk '{print $1, $4}' | awk '{bytes[$1] += $2} END {for (ip in bytes) print bytes[ip], ip}' | sort -rn
145048 229.212.21.212
89048 247.205.80.23
88981 137.216.107.80
64852 88.171.101.239
62291 224.58.7.136
57802 175.134.52.113
57760 143.110.35.240
52181 23.101.47.99
51233 171.233.106.117
45395 77.255.37.60
37809 177.132.224.221
30623 184.238.0.32
28144 50.199.204.183
```

Verified Answer: "`229.212.21.212`"

9. Challenge Question 8: How many total bytes were sent by the above IP address that sent the most amount of data? (10 Points)

The same methodology and commands from the previous question also give us the answer to this question (answer = the ***value*** on the left side of the first line).

Verified Answer: "`145048`"

10. Challenge Question 9: What was the busiest day (day with the most bytes transferred)? (Answer format is YYYY-MM-DD) (20 Points)

We are back to our **entries.txt** file that we created from the file body in section 1. Column 3 is the date/time stamp (in decimal format) and column 4 is the transferred bytes for each log entry. We need another associative array to sum up transferred bytes by date. But first we need to convert the integer timestamps to UTC dates.

When converting dates, you have to tell Linux if you want UTC dates/times vs local dates/times. It will often default to local dates/times, which would give the wrong answer to this question.

First, we **cat** the file to create a stream for piping commands. Then we use **awk** to pull out columns 3 and 4. At this point, for the next **awk** command, we only have two columns! Effectively, we screened out columns 1 and 2, so now column 3 becomes column 1 and column 4 becomes column 2.

However, the second **awk** command is *processing* data, not filtering it. It is reassigning the converted date string back to column 1, overwriting the integer decimal value. We still have our two columns, but we have *transformed* column 1. We use the **strftime()** function to convert the decimal date/time stamp to a string of text in YYYY-MM-DD format (we don't need or want the time portion). The **$1** parameter is the data from column 1, and the other **1** parameter tells the **strftime()** function to convert to UTC, *not local*. Then we **print** the stream (both column 1 and column 2) to standard output and **sort** it. It will sort by date. Then we use the redirection operator **>** to send the data stream to a new file called **dates.txt**.

Finally, we run **dates.txt** through an associative array to set up our *key-value pairs* with dates for keys and bytes for values. We sum the bytes by date, just as we did with IP addresses in the previous two questions.

```
nicegoat@Goatyard:~$ cat dates.txt | awk '{bytes[$1] += $2} END {
for (dts in bytes) print bytes[dts], dts}' | sort -rn | head -5

65676 2018-03-23
57942 2018-03-26
57155 2018-03-10
49399 2018-03-18
45243 2018-03-27
```

Verified Answer: "2018-03-23"